

Metrion

Realistic enough for what: the vocabulary of a taxonomy that refuses to score an environment on its own

Every claim about an autonomous cyber agent is a claim about the environment it was measured in, and the field has no shared way of saying what that second claim amounts to. Papers describe an environment by the features it implements, or by a broad assertion of realism, and neither tells you whether the environment supports the thing you want to say with it. Metrion is the attempt to replace the adjective with a procedure.

This file is the vocabulary that procedure needs. Four method words, eleven dimension names, ten scoring words, a second layer of scenario terms that are not dimensions and keep being mistaken for dimensions, and an ontology of environment kinds that exists only because the project spent months using simulator, emulator, range and benchmark interchangeably and then had to stop.

Poster at ACM CCS 2026 with Maria Rigaki and Carlos A. Catania, *Realistic Enough for What? Metrion: A Multidimensional Framework for Evaluating Cyber Environments*. Interactive scorecard at <https://stratosphereips.github.io/realism-framework/>. The comparison across published environments currently uses dimension-level proxy grades rather than element-level scoring and has not been validated, which is a fact about the evidence and is stated on the page rather than hidden in a limitations paragraph.

The word *environment* carries six senses across my files and they are counted in `dict_cyber_env`. Read that one first if the noun in any sentence below is doing more work than it looks like.

Contents

1	The question the project renames	3
2	The failure it fixes	4
2.1	Three audits, a decade apart, same verdict	5
3	The method, four words	5
4	The eleven dimensions	7
4.1	Infrastructure	8
4.2	Organizational behaviour	9
4.3	Security layer	10
4.4	Agent interface	10
4.5	External context	11
5	The scoring vocabulary	12

6	What it shows	13
7	The second layer	15
8	The ontology, and why it exists	16
8.1	Three tiers, and what may be compared with what	18
9	Use cases	19
10	Validating the framework	19
11	What stays elsewhere	21
12	Next	21

1 The question the project renames

realistic for what

*“So the answerable question is not whether an environment is realistic, but realistic **for what**.”*

website, When We Say “A Realistic Cyber Environment”, why “is it realistic?” is the wrong question

Definition: The reframing the whole project rests on. Realism is a relation between an environment and an evaluation objective, and no property of the environment alone answers it.

Distinguishes it from: Fidelity ranking, which asks which environment is closest to reality and produces an ordering that mostly measures implementation category. Also from feature listing, which answers a question nobody asked: what is in there, rather than what can be claimed with it.

Operational test: Can the sentence be completed with *for*? If somebody says an environment is realistic and cannot name the objective, there is no claim on the table yet.

In my project: The title of the page, the thesis of the poster, and the reason the output is a pair rather than a score. It is also the sentence that separates this from the two prior environment taxonomies, which classify by features implemented rather than by what techniques require.

partial confirmation

“Verification and validation of a model of a natural system are impossible in principle, because such systems are never closed and model results are never unique. What remains available is partial confirmation, in relative terms, against what is already trusted.”

website, same section

Definition: The epistemological position underneath the reframing. A model of an open system cannot be verified or validated in the strict sense, so the strongest available result is confirmation against something already accepted, and always in relative rather than absolute terms.

Distinguishes it from: Validation as the modelling and simulation literature uses it, which presumes a closed system and a unique correct answer. Also from fidelity in the 2013 MITRE sense, which is consistency between a represented feature and its real counterpart from some perspective and at some level of detail, and is a narrower and more tractable notion than validity.

Operational test: Name the thing the environment is being confirmed against. If the answer is reality, the claim cannot be made. If the answer is a technique catalog, a documented campaign, or an operator’s account, it can.

In my project: The argument that licenses replacing verification with a requirement profile. It is also why the whole framework is derived from ATT&CK and D3FEND rather than from expert intuition: those catalogs are the already trusted thing, and the derivation is checkable against them.

fidelity, in the 2013 sense

Definition: Consistency between a represented feature of an environment and the corresponding feature in the real world, viewed from some perspective or at some level of detail. Distinct from concreteness, which is how detailed the representation is, and from how completely it covers the

possible alternatives. A representation can be highly detailed and still low fidelity if its content does not match.

Distinguishes it from: Realism in this framework, which is scored per dimension and relative to an objective. Fidelity is a property of a single represented feature and says nothing about whether that feature matters for the claim. The 2013 paper also records that fidelity for technical aspects is easier to determine than for operational and threat aspects, which is exactly the split this framework’s dimension groups reproduce.

2 The failure it fixes

Two pictures, one from each side of the engagement, and they are the same picture.

“Picture a defender benchmark where attacks raise alerts but ordinary enterprise life is thin, few normal logins, little admin maintenance, hardly any background traffic. A defender scores well there because malicious behaviour is being read against a clean baseline.”

website, the failure it fixes

“In NASimEmu an action can succeed deterministically in the simulator and fail against the corresponding real service in the emulator, because the simulator abstracted away the service and operating-system fidelity that decides whether an exploit lands. PenGym reports the same gap.”

website, the failure it fixes

“So the score reflects the agent’s capability and which conditions the environment modelled or omitted, with no way to separate the two.”

website, the failure it fixes

That last sentence is the defect the framework exists to make visible. It is a confound, and it runs in both directions: an environment can flatter an agent by removing the thing that would have defeated it, and it can defeat an agent by omitting the thing it needed. Five worked cases of the first kind sit in the project notes and each names the dimensions that produce it: an empty enterprise with no benign activity, so detection is anomaly-spotting against nothing; clean structured alerts with no duplicates or delays, so triage becomes label-following; credentials that behave like global keys rather than scoped and expiring artifacts; a known-vulnerable target with a clean exploit path; and real shell access inside a world with no logs, no users and no egress controls.

gain attribution

*“**Gain attribution** is improvement that stems from information or capability engineered into the environment rather than from the learned policy.”*

website, gain attribution

Definition: As stated. A reported improvement whose cause is a property of the environment rather than of the method being credited for it.

Distinguishes it from: Overfitting to a benchmark, which is a property of a trained policy and can be detected by holding data out. Gain attribution is a property of the setup and survives every held-out split, because the information was engineered into the environment for both training and test.

Operational test: Two, and the page names both: evaluate a random-action baseline in the same environment, and compare approaches under equivalent state and action spaces. If the random baseline is respectable, the environment is supplying the gain.

In my project: The recurring mechanism behind the three audits, and the term that connects this project to the deployment-readiness question in `td_shared`. A policy that has learned the benchmark and an environment that hands the policy its answer are the same failure seen from opposite ends.

The sharpest illustration comes from outside security, and it is worth carrying because it is the version nobody argues with. On one widely used tool-use benchmark a trivial agent returning empty responses scores 38% and outperforms a frontier-model agent, because impossible tasks are graded as successes. Issues of this kind move reported capability by up to 100% in relative terms, and 24% of the top fifty leaderboard positions on a prominent coding benchmark are wrong. Cost is the second unmeasured axis, and ignoring it has made the best-performing agents needlessly complex.

2.1 Three audits, a decade apart, same verdict

Security machine learning has been audited three times and each audit reached the same conclusion about its own generation. Supervised security learning: ten pitfalls. Large-language-model security research: nine pitfalls, present in every one of 72 papers at leading security and software-engineering venues, with only 15.7% of instances explicitly acknowledged. Deep reinforcement learning for cybersecurity: eleven pitfalls across 66 papers from 2018 to 2025, averaging 5.8 per paper, of which 71.2% show no evidence of policy convergence, 66.7% neglect variance analysis, 60.6% leave partial observability unmitigated, and 40.9% evaluate in oversimplified or contrived environments.

“the pitfalls obscure whether a reported improvement comes from genuine algorithmic advance or from artifacts of simplified environments and incomplete evaluation”

website, three audits, a decade apart, same verdict

The pattern is not security-specific. Benchmarking is the primary form of experimentation in 91% of empirical reinforcement-learning papers, and doing it well enough to support the claims made carries computational costs that are often prohibitive, which is an argument for an additional experimental paradigm rather than for better benchmarking alone. Assessed against 46 best practices, widely used AI benchmarks show large quality differences, most report no statistical significance, and most cannot easily be replicated.

3 The method, four words

All four are Active. They are the four boxes of the methodology figure and the order matters, because each one constrains what the next is allowed to invent.

Derive

“Start from a use case, decomposed on both sides, attacker goal, technique, procedure; defender posture, detection priority, countermeasure. Walk MITRE ATT&CK and D3FEND, recording what each technique requires the environment to provide, model, or emit.”

website, the method

Definition: The bottom-up pass. For every technique in scope, ask three questions: what must the environment provide for this to be attempted, what must it model for the outcome to be determined, and what must it emit for the outcome to be observable. The answers are requirements, not dimensions.

Distinguishes it from: Brainstorming a dimension list, which is what prior environment taxonomies did and which cannot be shown wrong. This can: if a dimension is required by no technique it does not belong in the catalog, and if a technique requires something no dimension covers the catalog is incomplete.

Operational test: Can every dimension be traced back to named technique identifiers? The two derivation traces exist for exactly this, one file for the attacker-side ATT&CK pass and one for the defender-side D3FEND pass, and they feed one unified catalog.

In my project: The step that supplies the credibility, because ATT&CK is already the security community’s vocabulary and a framework built on it inherits adoption rather than competing for it. It is also the step with the deepest dependency: the catalogs are themselves curated abstractions, which is what forces the fourth step.

The two sides of the decomposition use different top-layer vocabulary and the difference is not stylistic. On the attacker side ATT&CK gives tactic, then technique, then sub-technique, then procedure, collectively TTPs, with group and campaign above them as attribution objects. Those tactics are unordered, and only the Lockheed Martin kill chain is a strict ordered progression, so nothing in this framework may claim the tactics are a sequence. On the defender side D3FEND gives defensive tactic, from the set Model, Harden, Detect, Isolate, Deceive, Evict and Restore, and countermeasure, linked to ATT&CK through a digital artifact ontology rather than by direct mapping. D3FEND deliberately carries no priorities and does not prescribe or rank countermeasures, so the layer above it, defensive posture and detection priorities, is threat-informed-defense vocabulary and not D3FEND’s.

Cluster

“Those per-technique requirements converge into eleven realism dimensions, in five groups”

website, the method

Definition: Pooling the requirements from both passes and merging them until what remains is irreducible. A dimension is irreducible when removing it makes at least one class of techniques, attacker or defender, untestable.

Distinguishes it from: A taxonomy of features, which groups by what is implemented. This groups by what would break, which is why two dimensions can correlate strongly in practice and still be separate: emulation platforms bundle OS with action realism and defender stacks bundle telemetry with defensive controls, and those are implementation habits rather than conceptual overlap.

Operational test: Delete the dimension. If nothing becomes untestable, it was not a dimension.

In my project: The step that produced the eleven, and it has already been run more than once. The catalog started at fourteen and was reduced: physical realism and cloud or virtualization fidelity were moved out into parallel frameworks scored separately, and organizational and trust realism was folded into identity, for federation and delegation, and into external ecosystem, for third-party infrastructure.

Score

“For a given objective, each element is critical, useful, or not needed. Coverage of each is full, partial, absent, or unknown.”

website, the method

Definition: Two independent judgments applied to the same list. The objective decides what it needs, the environment decides what it has, and the score is a function of both.

Distinguishes it from: A single coverage percentage, which asks only the second question and therefore cannot distinguish a bad environment from a good environment used for the wrong thing.

Operational test: Change the objective without touching the environment. If the number does not move, this is not scoring in Metrion’s sense.

In my project: The step that makes the framework actionable rather than descriptive, and also the step with the largest exposure, since both judgments are made by a person reading documentation and code.

Validate

“ATT&CK and D3FEND are themselves curated abstractions, so the dimension set needs outside checking”

website, the method

Definition: Semi-structured interviews with practitioners in academic, SecOps and pentesting roles, followed by a broader survey, asking what to add, remove, split or merge and whether people agree on what matters for a given objective.

Distinguishes it from: User testing of a tool, which would ask whether the scorecard is usable. This asks whether the heuristic set is right, which is a question about the framework’s content and not about its interface.

Operational test: Two questions, and they are the poster’s own: which dimensions would practitioners add, remove, split or merge, and do practitioners agree on each dimension’s importance for a given objective. The first is about the set, the second about the profile.

In my project: The step that has not been run. It is designed rather than done, and Section 10 is what the design currently commits to.

4 The eleven dimensions

Each dimension is Active. Each entry gives the definition, the evaluation question the poster table puts against it, what its absence makes untestable, and where it lands across the thirteen environments scored so far. Five groups, and the grouping is by which part of an evaluation the dimension affects rather than by which layer of the stack it sits on.

4.1 Infrastructure

topological realism

Definition: The structure of the network, how hosts reach each other, how segmentation is enforced, and what is observable from a given vantage point. Not only the graph: also the protocols, devices and trust boundaries that give the graph its meaning.

Evaluation question: Do the network, services and hosts resemble those of a real technical system?

Absence makes untestable: Anything where reachability is the mechanism. Segmentation-dependent lateral movement, position-dependent sniffing, staging and egress asymmetry, and the discovery problem itself, since a topology handed to the agent is not a topology it had to find.

Where it lands: Partial almost everywhere and full nowhere. It is the dimension abstract simulators do best and still none of them scores full, because what they model is the graph rather than the enforcement.

service fidelity

Definition: The application-layer daemons running on top of the operating system and the network: what they expose, how they respond, what versions and vulnerabilities they carry, and what data they hold. After the decomposition, operating-system subsystems and identity infrastructure live in their own dimensions, so this is specifically about daemons and the content they serve.

Evaluation question: Are the services real, versioned, fingerprintable, and populated with content worth taking?

Absence makes untestable: Exploitation against a real service stack, and every objective whose target is data rather than access. If the file shares hold nothing, there is nothing meaningful to exfiltrate.

Where it lands: The clean split. Real-software emulators score full, abstract simulators score absent, and there is almost nothing in between.

operating system fidelity

Definition: The layer underneath application services: the autostart and scheduling surface, execution-flow and loader behaviour, process and memory and privilege primitives, configuration stores and filesystem semantics, the kernel and code-signing trust layer, credential material at rest and in memory, and the event-generation paths that make all of it observable.

Evaluation question: Do operating-system subsystems behave as real subsystems, attackable and instrumentable, on whichever platform the scenario requires?

Absence makes untestable: Persistence, privilege escalation, living off the land, anti-forensics, and every destructive impact primitive. It is the largest technique block in the derivation and the one abstract simulators score zero on.

Where it lands: Full for the emulators and the capture-the-flag platforms, absent for every abstract simulator. The Windows emphasis in the element list reflects where ATT&CK and endpoint telemetry are most developed, not a claim that Windows is the only platform in scope.

4.2 Organizational behaviour

identity realism

Definition: Who can authenticate as whom, under what protocol, carrying what material. Populated directories, credential material across its full lifecycle, authentication protocols with their on-the-wire behaviour, federation and token infrastructure, multi-factor authentication as a design space, and the tiered administration model.

Evaluation question: Does the environment contain realistic users, credentials, and the authorization relationships between them?

Absence makes untestable: Credential-based lateral movement as anything other than graph traversal. This is the dimension that decides whether a stolen credential is a scoped, expiring, policy-bound artifact or a global key, and the difference is the difference between an evaluation and a puzzle.

Where it lands: GOAD is the strongest case in the catalog, multi-forest Active Directory with trust relationships, roastable accounts, certificate abuse and misconfigured group policy. The abstract simulators score partial at best, usually by way of an abstracted credential-reuse token.

temporal dynamics

Definition: How time is modelled. Whether events and state evolve on a real clock, whether delays and schedules and session lifetimes and token expiries and human rhythms exist as first-class phenomena, and whether the engagement horizon spans the days-to-months timescales real intrusions occupy.

Evaluation question: Does time pass, and does anything depend on it?

Absence makes untestable: Slow-drip operations, dwell, blending into business hours, persistence validated by surviving a reboot, and every claim where the defender's response speed is the thing being measured. Without it the time-pressure framing of ransomware collapses entirely.

Where it lands: Absent nearly everywhere. Turn-based stepping is the norm, and one early-stage environment is an outlier for modelling communication delays at all.

benign activity realism

Definition: The legitimate, non-attacker activity in the environment. Simulated users doing their work, services running their normal cycles, data being created and accessed, credentials being used through real authentications.

Evaluation question: Is there a baseline of normal for abnormal to be abnormal against?

Absence makes untestable: Every detection problem, every evasion scenario, and every technique that depends on material accumulated through use. Data-loss volume anomalies are anomalies against a baseline; with no baseline, covert exfiltration is not being evaluated on covertness at all.

Where it lands: The thinnest dimension in the catalog and the one I expect to be the reason defender evaluation does not work today. Two environments score partial, twelve absent, none full. The literature agrees: a sixteen-year systematic review of a hundred cyber-range papers lists four user-behavior generation tools in total.

4.3 Security layer

defensive realism

Definition: The countermeasures present in the environment and how they behave. Preventive controls that block, detective controls that alert, hardening configurations that raise the cost of a technique, and response controls that act back on the attacker, together with their imperfection and their tamperability.

Evaluation question: Are there controls, do they miss, and can they be attacked?

Absence makes untestable: Evasion, containment, and any claim about a defender's decisions rather than a defender's presence.

Where it lands: Universally weak. No environment scores full. The strongest are partial, and the ceiling is set by a scripted blue agent with a small action space.

Boundary: This dimension covers mechanisms and response primitives, not the strategic richness of a blue actor. Whether the defender is absent, scripted, policy-driven, adaptive, learning or human is a scenario-level property and lives in Section 7. That boundary was not always drawn, and the box below records where it failed.

telemetry fidelity

Definition: The data-generation process for security-relevant events. Whether the environment emits realistic logs, events, flows and artifacts as a byproduct of activity, and whether those signals survive collection, transport, normalization and retention to reach somewhere a defender can query them.

Evaluation question: Does the environment generate the evidence, at real schemas, real volume and a real noise floor?

Absence makes untestable: Triage, hunting, incident reconstruction, and any claim about analyst workload. The decisive element is the one that says security-relevant events must be generated as a byproduct of activity rather than injected as oracle outputs.

Where it lands: Absent nearly everywhere, with one simulator the exception by way of a security-information-and-event-management integration. Together with benign activity this is the pair whose absence rules out the entire defender-side use-case column.

4.4 Agent interface

action realism

Definition: What an agent can actually do, at what level of abstraction, with what parameter space, and with what consequences when it fails. Scores well when the agent's interface is the one a human operator would use, a shell, a tool, an API, and the environment responds with what the real platform would produce.

Evaluation question: Is the action space the real one, or a menu?

Absence makes untestable: Anything about tool use, composition, or debugging under failure. The gap between picking from a twenty-element menu and composing a real call with correct

parameters and credentials is the gap between a toy and an evaluation.

Where it lands: Full for adversary-emulation tooling, the emulators and the capture-the-flag platforms; absent for the abstract simulators. The element that does the most work is failure realism: failure as the deterministic consequence of a real mismatch rather than a probability draw, so an agent can read the error and adapt.

Defender side: The defender action space has to distinguish families with genuinely different targets and scoped effects: hardening, credential operations, process eviction, isolation and mediation, and restoration. Collapsing them into one generic respond action makes defender tradecraft unevaluable, and multi-step operations such as account recovery have to walk their intermediate states rather than resolve as a super-action.

observation realism

Definition: What an agent perceives when it queries the environment. Scores well when what you see depends on where you are, what you asked, when you asked, and with what privilege, and when understanding has to be assembled over many observations and is never complete.

Evaluation question: Is the agent given the state, or does it have to look?

Absence makes untestable: Reconnaissance as a decision problem, and every defender claim that requires reconstructing compromise from evidence.

Where it lands: Partial by design in the abstract simulators, which discover nodes incrementally, and full in the emulators, where the interface is a real tool returning real output.

The sharpest element: The oracle-free property: the agent is never handed ground-truth state, and everything the evaluation depends on must be derivable from observations the agent could itself make, with no leakage through reward shape, action mask or state vector that an operator could not obtain. Most reinforcement-learning environments fail it by construction, and it is the single clearest line between realistic and abstract.

4.5 External context

external information ecosystem

Definition: The world beyond the target network. Public sources, DNS and certificate transparency, code repositories and social media, attacker-controlled domains and hosting, cloud and content-delivery services used for blending, compromised third parties, and the software supply chain. Split into a passive sub-dimension, what the attacker reads from, and an active one, what the attacker builds, rents or abuses.

Evaluation question: Is there a multi-party world, or does the model stop at the perimeter?

Absence makes untestable: The entire reconnaissance and resource-development tactics, most of initial access, and all of command and control. Also the destination side of any exfiltration claim.

Where it lands: The most uniformly missing dimension in the catalog. No environment models external DNS, open-source intelligence, command-and-control infrastructure, or supply chain.

Asymmetry: This dimension is attacker-driven by construction. D3FEND does not enumerate it as a defender requirement, because its techniques target the enterprise interior, and the

defender’s connection to it is observational: external scanner indices, reputation feeds, certificate-transparency monitoring, federated-identity audit, decoy domains. Those uses are captured in the dimensions that carry them rather than duplicated here.

5 The scoring vocabulary

scoring element

“Each is operationalised through concrete scoring elements, 115 of them, set at the level where a missing property makes a class of techniques inexpressible or unobservable.”

website, the method

Definition: The unit that is actually scored. A dimension is a heading; an element is a checkable property, pitched at the level where its absence removes a class of techniques rather than an individual variant of tradecraft.

Distinguishes it from: The dimension, which is the cluster the elements belong to, and from a technique identifier, which is the derivation anchor and may appear against several elements.

Operational test: Ask what stops working if this is missing. If the answer is one specific procedure, the element is pitched too low. If the answer is nothing, it is not an element.

In my project: The level the framework is designed to score at and the level the published comparison does not yet use. That gap is stated on the page, and it is the difference between eleven judgments per environment and 115.

requirement profile

Definition: The objective’s half of the score. Each element marked critical, useful, or not needed for a stated evaluation objective, before any environment is looked at.

Distinguishes it from: Coverage, which is the environment’s half. Keeping the two authored separately is what stops the profile being written to fit the environment in front of you.

critical, useful, not needed

Definition: The three-level requirement scale. Critical means the objective cannot be meaningfully evaluated without the element. Useful means it contributes but is not the bottleneck. Not needed means it is incidental to this objective. Critical weighs 2 and useful weighs 1.

Distinguishes it from: A severity scale in a usability inspection, which grades problems found. This grades requirements before anything is inspected. Whether three levels are enough is one of the three questions the poster puts to its readers.

full, partial, absent, unknown

Definition: The four coverage grades, assigned from an environment’s papers, documentation and code. Full counts 1, partial 0.5, absent 0. Unknown is excluded from the average and reduces a separate completeness measure instead.

Distinguishes it from: The requirement scale above, which is about the objective. Keeping unknown as unknown rather than assuming absent is a deliberate choice: it makes the table look less clean and keeps it honest about under-documented and closed environments.

fit score

“The weighted average is a fit score from 0 to 1.”

website, the method

Definition: The weighted average of coverage values over the required elements, weights taken from the requirement scale. Zero means nothing the objective requires is present, one means every requirement is fully met.

Distinguishes it from: Technique coverage, which is the fraction of the techniques requiring a dimension that are actually testable in the environment. The two measure different things and legitimately diverge: an environment can model a dimension shallowly and still support none of its techniques, so it can score partial on realism and zero on coverage in the same row.

Operational test: Did the objective change the weights? If the same number comes out for two different objectives, the profile was not authored.

In my project: The headline number in the worked example, and the number that is explicitly not the verdict. The verdict is the hard constraint below.

the hard constraint

“if any critical requirement is absent, the environment is not suitable for that objective”

website, the method

Definition: The rule that overrides the fit score. One missing critical property can invalidate the evaluation regardless of how high the average is.

Distinguishes it from: A threshold, which is a statement about degree. This is a statement about kind, and it is what stops the framework being a ranking.

Operational test: Is any critical element graded absent? Then the verdict is fixed and the average is decoration.

In my project: The reason a 0.39 and a 0.88 on the same environment are not two points on a scale. One of them also failed five critical requirements.

suitable, partially suitable, incomplete

Definition: The three verdicts once the hard constraint has been applied. Suitable above a threshold set at 0.75, partially suitable below it, and incomplete when too much of the profile is unknown. The weights and the threshold are working values and are stated as such.

Distinguishes it from: The fit score, which is continuous. Incomplete is the verdict that has no analogue in the score at all: it says the evidence about the environment is too thin to reach a verdict, which is different from a low one.

6 What it shows

*“Applied to GOAD, an emulated multi-domain Active Directory environment: **suitable** for credential-based privilege escalation at a fit of 0.88, and **not suitable** for targeted data exfiltration at 0.39 with five critical requirements unmet, the same environment, unchanged.”*

website, what it shows

“Suitability is a property of the pair, not of the environment.”

website, what it shows

The two panels of the worked example are radar charts in which the dashed polygon is the objective’s requirement and the solid polygon is the environment’s coverage, and a dimension is met when the solid reaches the dashed. For credential-based privilege escalation the critical requirements are service, operating system, identity, action and observation realism, and GOAD covers all five fully; topology and temporal are left partly unmet and are only useful. For targeted exfiltration the requirement extends along the context axes, where temporal, defensive, benign activity, telemetry and external ecosystem are all critical, and GOAD is absent on every one, so the solid polygon collapses toward the centre exactly where the objective needs it to reach.

the two families

“Real-software emulators reproduce service, operating-system and action-level realism. Abstract simulators reproduce topology and the agent interface but abstract away temporal dynamics, defensive controls, benign activity and telemetry.”

website, what it shows

Definition: The field-level result. Across thirteen publicly inspectable enterprise environments, realism splits into two clusters that do not overlap, and the split follows implementation rather than effort.

Distinguishes it from: The claim that environments are not realistic enough, which is the usual complaint and is weaker. They are realistic in different, non-overlapping ways, and the environments where agents can act realistically are the ones where defenders cannot operate realistically.

Operational test: Ask which family the environment is in, then predict its weak dimensions. If the prediction holds, the split is doing the work rather than the individual scores.

In my project: The finding, and the reason the framework is worth having even before element-level scoring. One simulator sits partly between the two, by investing in green agents and event-management integration, which shows the split describes current practice and not a structural limit.

“Those context dimensions are the least represented anywhere, which means the objectives that depend on them, including targeted exfiltration, evasive operations and threat hunting, are currently supported by no environment at all.”

website, what it shows

That sentence is the strongest claim on the page and it is a claim about a gap in the field rather than a deficiency in any tool. Threat hunting for long-dwell actors is the sharpest case: it needs benign activity, rich telemetry and multi-week temporal dynamics simultaneously, which is the most demanding combination of exactly the three dimensions nobody has built.

7 The second layer

Three things the eleven dimensions do not capture, and none of them is a twelfth dimension. They sit above the catalog as a separate layer, and the distinction is the one methodological point most likely to be lost when this work is summarized by someone else.

environment realism against scenario validity

Definition: Two layers. Environment realism is what the environment can represent, and it is the eleven dimensions. Scenario validity is whether a given scenario uses those capabilities in a way that makes the claim meaningful. An evaluation is defensible only when both hold.

Distinguishes it from: A single realism score, which conflates them. An environment can be detailed and faithful and still be run with a scenario no real attacker would have: full prior knowledge of the network, no time pressure, and an objective like compromise every host. The environment is realistic and the scenario is not, and the conclusions do not travel.

Operational test: Two questions in order. Does the environment provide the capabilities the objective requires? Does the scenario connect them through persistent state, causal progression, plausible timing, consequences and coherent evidence? A no to the first is a missing dimension. A no to the second is a valid environment being misused.

In my project: The organizing result of the notebook's last four questions, and the reason the catalog was not extended when three genuine gaps turned up.

failure-mode realism

Definition: Whether the modelled capabilities fail, degrade, interact and produce side effects the way real ones do. Do not ask only whether services exist, ask whether they crash, return partial results, expose inconsistent versions, and fail under load. Do not ask only whether an exploit action exists, ask whether it can partially succeed, corrupt state, trigger detection and leave traces that change the next step.

Distinguishes it from: Any single dimension, since it cuts across six of them. It is the failure-mode-and-effects question applied to an environment rather than to a product, and it is a property of how each dimension is implemented rather than of whether it is present.

campaign consistency

Definition: Whether a scenario connects its techniques into one campaign. Six properties: goal continuity, so actions serve an objective rather than instantiate a checklist; state continuity, so credentials and footholds and discoveries and alerts persist and change later possibilities; causal continuity, so later actions are explainable from earlier ones and the attacker cannot use what it never obtained; temporal continuity, so phases run at plausible and different timescales; cost and risk continuity, so loud or destructive actions are not free; and trace continuity, so the evidence forms one correlatable story.

Distinguishes it from: The dimensions it depends on. State continuity needs identity, action, observation and temporal; trace continuity needs telemetry, observation, identity and benign activity. The point is not to add axes, it is to require that the axes compose. Break any of the six and the agent may still solve individual tasks while campaign success stops meaning campaign success.

strategic interaction, and the opponent-model scale

Definition: A requirement attached to a use case rather than to an environment: what kind of opposing actor exists, what each side can observe, whether either adapts, and how one side's actions change the other's options. Recorded on a six-level scale. Level 0, no opposing actor. Level 1, static environment effects, where controls or traces exist but nobody decides anything during the run. Level 2, a scripted opponent. Level 3, a policy-driven opponent that chooses on observations without adapting its strategy. Level 4, an adaptive opponent that changes behaviour on observed actions, failures or inferred intent. Level 5, co-adaptive or human, where both sides adapt.

Distinguishes it from: Defensive realism, which covers mechanisms. The scale covers the actor. The distinction is what resolves an asymmetry the framework had for months and the box below records.

One dimension doing two jobs, and only when the attacker is the learner

Defensive realism was conflating defender *implementation*, the sensors and controls and tooling, with defender *behavioural richness*, the blue action space and policy.

While the acting agent is the attacker that conflation is invisible, because the one dimension happens to capture opposing-actor quality. Turn it around and the gap opens: when the defender is the learner, the attacker is the opposing actor and no dimension measures its richness at all. Action realism is the acting agent's, external ecosystem is attacker infrastructure, and neither says whether an embedded red agent adapts. So a defender environment with a scripted attacker is less realistic for defender objectives than one with an adaptive attacker, and the eleven-dimension scoring cannot see the difference.

The resolution is not a twelfth dimension. It is the opponent-model scale above, applied symmetrically to whichever side is opposing, with each use case stating its minimum level. Worm spread wants level 1 or 2. Evasive operation wants at least 3 and often 4. Long-dwell hunting wants 3 or 4. Attack-defense co-evolution is level 5 by definition.

8 The ontology, and why it exists

Working tier. These are the words the project used interchangeably for months, and the fix is not better definitions of each but noticing that they were never on one axis.

implementation against usage

Definition: Two independent axes. Implementation says what the environment is built out of. Usage says how it is packaged or used to evaluate. Most real environments are a pair, one from each, and asking whether something is a cyber range or an emulator is a category error because it can be both.

Distinguishes it from: A single taxonomy of environment types, which is what every prior survey offers and what forced the interchangeable usage in the first place. Metrion cares about implementation, because the eleven dimensions ask what the environment can represent. Usage decides what kind of evaluation it supports, scored task or agent training or human exercise or dataset generation, and not what realism it can deliver.

simulator

Definition: A programmatic model of a network, hosts and agent actions in which services, operating-system behaviour and exploit outcomes are abstractions: typed services, probabilistic exploit success, symbolic state vectors. Trades fidelity for speed and scale, which is what buys the millions of steps reinforcement learning wants.

Distinguishes it from: An emulator, by the absence of a real operating system and service layer. Nothing else in the definition is decisive, and in particular being fast is a consequence rather than a criterion.

emulator

Definition: A network of real software components, containers or virtual machines running real operating systems, services and security tools, wrapped in a controlled topology and exposed through a programmatic agent interface. Agent actions become real system calls and exploits are real tool invocations.

Distinguishes it from: A simulator, as above, and internally by substrate: container-based emulators cannot host kernel-level adversary behaviour, rootkits and bootkits and anything below the kernel, while virtual-machine-based emulators close that gap. Hardware-inclusive variants extend it further at significant cost and are scored as a separate descriptive property rather than as a third implementation kind.

cyber range

Definition: A platform for delivering interactive environments for training, exercise, test and evaluation, or posture testing. What makes it one is the packaging: team formation, a scenario library, an instructor interface, scoring, and repeated or persistent runs.

Distinguishes it from: An implementation kind. It is orthogonal, and a range may sit on a container emulator, a virtual-machine emulator, a commercial hybrid, or an emulator plus hardware. The authoritative definitions converge on the training-platform reading and explicitly allow hardware without requiring it, so an earlier working definition here that made hardware definitional was tighter than the literature supports.

benchmark

Definition: A curated, scored collection of tasks used to compare agents against a fixed rule. What makes it one is the fixed task set plus an objective scoring rule.

Distinguishes it from: Both implementation kinds, since it is orthogonal to them: a benchmark may run on a simulator, on emulated or real-infrastructure capture-the-flag boxes, or on a range.

testbed

Definition: Research infrastructure used primarily to generate datasets or study phenomena rather than to score agents.

Distinguishes it from: A benchmark, by the absence of scored-task structure, and from a cyber range, by the absence of operational-defender completeness.

adversary-emulation platform

Definition: Software whose role is to execute attacker tradecraft against an arbitrary target and

report results. Its realism is whatever it is deployed against; the platform is not the environment.

Distinguishes it from: Everything above, and this is the entry that keeps the catalog honest. Many published evaluations of such a platform are really evaluations of the platform plus a particular emulator or range, and the scoring row inherits its operating-system and service realism from the deployment rather than owning it.

agent scaffolding

Definition: Prompt templates, tool wrappers, memory and planning loops that turn a language model into an agent acting against one of the implementation kinds above.

Distinguishes it from: The environment entirely. It is orthogonal to environment choice, since the same scaffolding plugs into a simulator or a capture-the-flag box, and confusing the two is how a scaffolding result gets reported as an environment result.

8.1 Three tiers, and what may be compared with what

the three tiers

Definition: Tier 1 is the filter: domain, implementation, and the use-case scope the authors claim. Tier 2 is descriptive and non-realism: scale, sides modelled, operating regime, hardware integration, scenario openness, evaluation purpose, license, determinism. Tier 3 is the eleven realism dimensions.

Distinguishes it from: A flat comparison table, which is what the first scoring pass produced. Realism scores are methodologically valid only within a Tier 1 cell: an attacker-focused abstract simulator, a two-sided virtual-machine emulator and a single-host capture-the-flag benchmark belong to different cells, and comparisons across them are mostly artifacts of that conflation.

Operational test: Do the two environments differ on domain, implementation or claimed use-case scope? Then a side-by-side realism score is misleading rather than merely unflattering, because they are answering different questions.

In my project: The correction that turned one table into a stratified one, and the answer to the question that prompted it: an enterprise simulator and an industrial control testbed are so different that scoring them on identical dimensions is not unfair, it is meaningless.

Two surveys, one colour, opposite roles

The cyber-range literature names a fuller set of team roles than red and blue, and two of the standard references use *yellow* for opposite things. In one, yellow is simulated normal-user behaviour and green is system administration and infrastructure operations. In the other, yellow is system administration and grey is background traffic generation.

So the colour does not survive the trip between two papers that are both about cyber ranges. Anything recorded on this axis has to carry the roles by name, admin operations, normal user, scripted attacker, scripted defender, autonomous team, and let the colour go. The count of sides modelled is not itself a realism axis; the quality of any modelled opposing actor is, and it belongs to the opponent-model scale rather than here.

9 Use cases

use case

Definition: A category of attacker or defender activity defined by what the agent is trying to accomplish and under what real-world constraints. The unit against which environments are evaluated. Labelled UC-A1 to UC-A9 for attackers, UC-D1 to UC-D6 for defenders, UC-S1 to UC-S3 for system-level.

Distinguishes it from: A scenario, which is one instantiation with a topology and an attacker script, and from an ATT&CK tactic, which is one adversary goal rather than an operation with a success criterion and a tension.

Operational test: Does it name a success criterion and a defining tension? Exfiltration trades collection thoroughness against detection probability; ransomware trades speed against coverage. A use case without a tension is a task list.

In my project: The second half of every pair, and the thing the requirement profile is authored against. Nine attacker cases, six defender cases, three system-level, of which five are retained for traceability and are out of scope for the current enterprise comparison.

group archetype

Definition: A pattern of attacker behaviour shared across multiple named threat groups, clustered by objective and operational style rather than by attribution. Eight of them cover the catalog: state espionage, financial cybercrime, ransomware operators, destructive sabotage, initial access brokers, supply chain operators, resource hijacking, and critical-infrastructure pre-positioning.

Distinguishes it from: A named group, which is an attribution object, and from a campaign, which is a documented intrusion set over time. The archetype is the level at which use cases are derived, so a use case cannot be dismissed as an academic construction and also cannot be defended by pointing at one actor.

unrealistic objective

Definition: An evaluation objective no real attacker or defender would hold. Compromise all hosts, maximize compromised nodes, find all flags, or an undefined defend the network.

Distinguishes it from: A missing dimension, which is an environment problem. This is a scenario problem and it is worse in one respect: an agent trained to take over everything will never learn stealth, prioritization or persistence maintenance, because none of those serve its objective. Real attackers compromise the minimum number of hosts that reaches the goal, and over-compromising raises detection risk for no gain.

10 Validating the framework

analytical evaluation method

Definition: The category Metrion belongs to: a set of heuristics plus a procedure by which expert evaluators, not users, inspect an artifact and predict whether it is fit for a purpose. Heuristic

evaluation and cognitive walkthrough are its siblings. The eleven dimensions are the heuristics, the 115 elements are the scorable form, critical and useful and not needed is a severity scale, and the fit score is an aggregation rule.

Distinguishes it from: An empirical evaluation, which would put agents in environments and measure outcomes. This predicts fitness by inspection, which is cheaper and carries a known failure mode.

Operational test: Who applies it, an expert or a user? An expert. That single answer places the method and imports its literature.

In my project: The reframe that reading the human-computer interaction methods manual produced, and it changes what practitioner validation means. There is an established procedure for building a heuristic set that captures consensus in an application area, involving several experts either jointly beforehand or by clustering afterwards what experts already developed. The interview study is the second variant and should say so.

the evaluator effect

Definition: The known failure mode of this method class: two trained evaluators applying the same heuristics to the same artifact disagree badly. In one study only about a fifth of problems were found by all four evaluators and nearly half by exactly one. Heuristic evaluation has been found the broadest and least reliable of eight analytic methods, precisely because what counts as consistent is left to the evaluator.

Distinguishes it from: Disagreement about the dimension set, which the interviews address. This is disagreement in applying a fixed set, and the full-partial-absent-unknown judgment has exactly this exposure. It implies a third study the plan did not have: a paired-rating study of the rubric itself, with the number of evaluators chosen from the standard detection-probability arithmetic.

the three studies

Definition: Interviews, a survey, and a paired-rating reliability study. Interviews buy realism, the survey buys generalizability, and the paired-rating study buys precision. Those three cannot be jointly maximized, which is the standard trade in method choice, and naming which each buys is the triangulation argument rather than a list of three activities.

Distinguishes it from: Running two studies and calling it triangulation. The checklist requires the same underlying question addressable by all methods, per-method biases named in advance, the bias mechanisms understood, the critical ones compensated, and discrepancies explained rather than averaged. That is five sentences in a methods section and it is what separates the two.

Two design decisions carry the most weight and both are about anchoring. The dimension list is the analysis frame, not stimulus material, so it should not be shown in the interviews at all: once a participant has seen it, nothing after it can serve as independent corroboration. And the participant supplies the case while the researcher does the abstraction, so a written objective is a comparison probe late in the session rather than the opening prompt.

There is also a population question that is not settled. The user of Metrion is not the same as a cybersecurity practitioner, and the candidates are different populations with different knowledge: the engineers who build environments and ranges, the researchers who train and evaluate agents in

them, offensive practitioners, defensive practitioners, and the people who decide whether to trust an evaluation result. My own argument for who to recruit is a lead-user argument, that practitioners are not using agents yet but will be, and adopting it means accepting the consequence out loud. The findings would describe the leading edge rather than current practice.

11 What stays elsewhere

`td_shared`: `sim2real` and `sim2sim`, the two-place reading of *realistic*, the reversed fidelity and realism dialects, transfer and its two objects, drift, out of distribution, and deployment-readiness.

`dict_cyber_env`: the six senses of *environment*, the census of forty-nine occurrences across the blog pages and the realism evaluation, the twenty places two passes disagree, and the six replacement phrases.

`td_training_in_realistic_environments`: Daedalus, the two backends, and the scoped statement of which respects are real, which is the filled-in instance of what this file defines in general.

The check that closes this file is the two-way one. Dimension names element and element names dimension. Requirement profile names coverage and coverage names requirement profile. Fit score names the hard constraint and the hard constraint names the fit score. Defensive realism names the opponent-model scale and the scale names defensive realism, which is the pair the framework got wrong once and the pair most likely to drift back together. Where a distinction runs one way only, one of the two definitions is wrong.

12 Next

Three questions, and the poster asks all three of its readers rather than answering them.

Whether the eleven dimensions and their decomposition into 115 elements are at the right granularity. Whether three levels, critical and useful and not needed, are enough to represent what an objective requires. And whether deriving requirements from ATT&CK and D3FEND, rather than cataloguing what environments implement, is a sound basis for comparison at all.

Underneath them sits the one I cannot put on a poster. The framework's completeness argument runs through the archetypes, and the archetypes are MITRE's curated abstraction of the groups, so if that abstraction is wrong or incomplete the use cases inherit the error and the dimension set inherits it after them. Every dimension traces back to a technique, which is the property that makes the catalog refutable, and it is also the property that makes it exactly as good as the catalog it was derived from.

The immediate work is smaller than any of that. Convert the coarse grades into per-element evidence tables for one abstract simulator, one emulator, and one real-infrastructure environment, with a link behind every judgment. Then the interviews, which are the only thing that can tell me whether practitioners recognize any of this.